



JavaScript

From 0 to Beginner

Start();

Variables

```
var variable1 = 1 // Método ambiguo, hoisting extraño
let variable2 = 1 // Variable Normal, limita el Scope
const variable3 = 1 // Variable que no puede ser modificada
```

Scope

```
let variable = 1;
function (){
    console.log(variable) // Permitido
}
```

```
function(){
    let variable = 1;
}
console.log(variable) ///! No se puede
```

For loop

```
let animales = ["perro", "gato", "loro"];  
Usando IN  
for (animal in animales){  
    console.log(animal) // → 0,1,2  
}
```

```
Usando OF  
for (animal of animales){  
    console.log(animal) // → perro,gato,loro  
}
```

Casting

Implicit Type Casting (sin especificar)

```
let conversion = "5" + 2 → 55
```

Explicit Type Casting (especificando)

```
let conversion = Number("5") + 2 → 7
```

Getting Element

```
document.getElementById() --> 1 Elemento, ID  
document.querySelector() --> 1 Elemento, Clase  
document.querySelectorAll() --> 1 Lista, Clase
```

Attribute Methods

`setAttribute()` → Modifica el valor de un atributo

`getAttribute()` → Obtiene el valor de un atributo

`removeAttribute()` → Elimina el atributo

Global Attributes

`objeto.classList`

`objeto.hidden`

`objeto.id`

`objeto.style`

classList

`add()` → Añade una clase

`remove()` → Remueve una clase

`toggle()` → Desactiva una clase

`replace()` → Reemplaza una clase

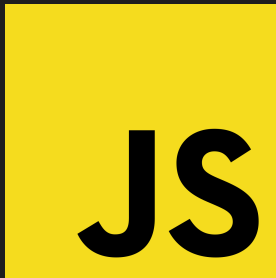
`contains()` → Verifica si el objeto tiene esa clase

Create

`createElement("etiqueta")` → Crea un elemento

`appendChild(padre)` → le asigna un hijo al elemento

`createDocumentFragment()` → En caso de querer crear muchos elementos, se crea una caja con todos los elementos (`documentFragment`) y después se añade la caja directamente al `html`



JavaScript

From **Beginner** to **Intermediate**
Course. `Start()`;

Window

`window.` ->

`open("link")` → Abre una ventana nueva

`close()` → Cierra la ventana actual o la del objeto

`closed` → **BOOL**: Verifica si la ventana se cerró

`name` → Obtiene la dirección de la ventana

`stop()` → Detiene la carga de recursos

`alert()` → Muestra un contenido de alerta

`print()` → Abre el menú para imprimir

`prompt()` → Abre un cuadro para insertar datos

`confirm()` → Abre un cuadro para poner Si o No

`minimize()` → Minimiza la ventana

EventHandlers and EventListeners

Events: son cualquier cosa que suceda en la página (click, mover el mouse, scroll, cargar)

EventListener: Captura el evento y hace algo con eso

Tipos de eventos / Event Handlers:

- click
- dblclick
- mouseover
- mouseout
- keypress
- keydown
- onkeyup
- select

Timers

`setTimeout(function, 2000)` → Lo puedes poner en cualquier parte del código y espera la cantidad de milisegundos indicada antes de ejecutar una función.

`clearTimeout()` → Termina con el timeout

`setInterval(function, 2000)` → Ejecutará la función en bucle cada vez que pasen la cantidad de milisegundos indicada

`clearInterval()` → Termina con el bucle

Switch

```
switch(variable){  
  case "caso1":  
    break  
  case "caso2":  
    break  
}
```

Callbacks

```
function hola(dato,dato2,cb){
    dato += 1
    cb(dato1,dato2);
}
```

Promises

Se usa para procesos largos como la lectura de un archivo, la descarga de un archivo, la conexión con una base de datos, etc.

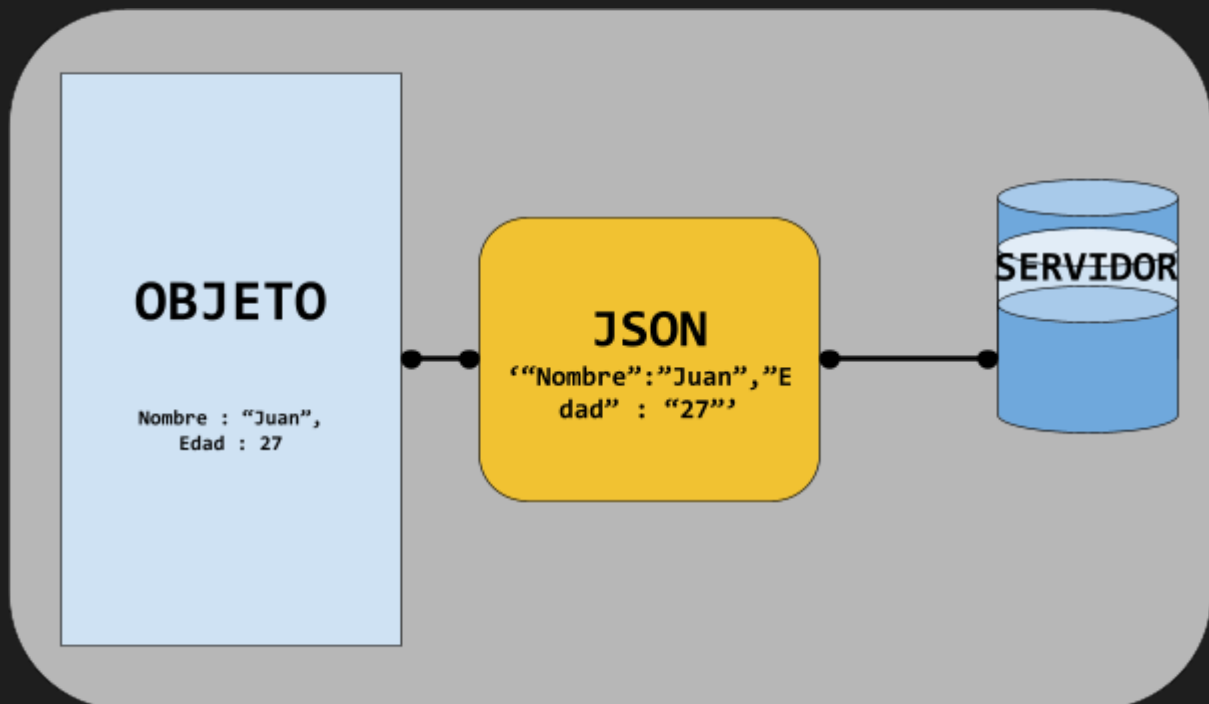
```
function pasearPerro(){
    return new Promise((todoBien, problema) => {
        let tengoPerro = true;
        let yaLoPasee = true;
        // Te prometo que te devolveré un valor, cuando termine
        if(!tengoPerro){
            problema("no tienes perro");
        }else if(yaLoPasee){
            problema("ya habias paseado al perro");
        }else{
            todoBien("Has paseado al perro");
        }
    });
};

// Si lograste pasear al perro bien, entonces.
```

```
pasearPerro.then(mensaje => console.log(mensaje))  
// Si algo sucedió  
.catch(error => console.error(error));
```

JSON

JSON (JavaScript Object Notation) es una manera de mandar datos al servidor en manera de objeto y que nos devuelva valores, el servidor solo recibe strings, así que para administrar variables, funciones u otros objetos tendremos que saber pasar las strings JSON a objetos y viceversa

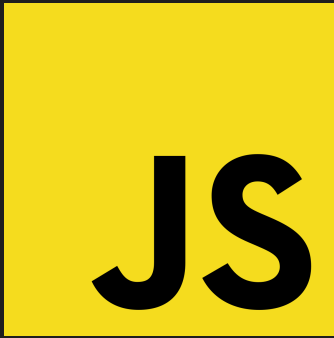


Objeto a JSON → `JSON.stringify(objeto)`
JSON a Objeto → `JSON.parse(objeto)`

Fetch

Con fetch nos encargamos de pedir datos al servidor y también mandar, sería la línea negra del dibujo de arriba.

```
// Devuelve una promesa (conectarte con la url)  
consulta = fetch("URL")  
// Si then significa que devolvió (Exito, Datos)  
Exito = consulta.then(response => response.json())  
// el también devuelve una promesa ya que los json tardan en pasarse.  
Exito.then(data => console.log(data));
```



JavaScript

From **Intermediate** to **Master**

Start();

Strict Mode

el strict mode es una instrucción para que el interpretador sea más estricto con la sintaxis del código

```
nombre = "máximo"  
console.Log(nombre)  
máximo
```

```
"use strict"  
nombre = "máximo"  
console.Log(nombre)  
error! variable no declarada (let,var,const)
```

Arrow Functions

manera más sencilla de hacer funciones

```
function hola(){console.log("hola!")}  
const hola = () => console.log("hola")
```

```
function hola(nombre){console.log("hola!" + nombre)}  
const hola = nombre => console.log("hola!" + nombre)
```

This contextual

la palabra this siempre hace referencia a su objeto padre (también funciones ya que son objetos), en caso de no tener objeto padre, por defecto será windows.

```
this === window
```

```
const objeto = {  
  nombre = "máximo"  
  saludar = function(){console.log("hola! soy" +
```

```
this.nombre)}
```

`this.nombre` → Undefined

`objeto.saludar()` → hola! soy maximo

Operador Ternario y Spread

```
(condición) ? accion true : acción false
```

```
(condición) ? (accion true, acción true) : acción false
```

```
// spread (...)
```

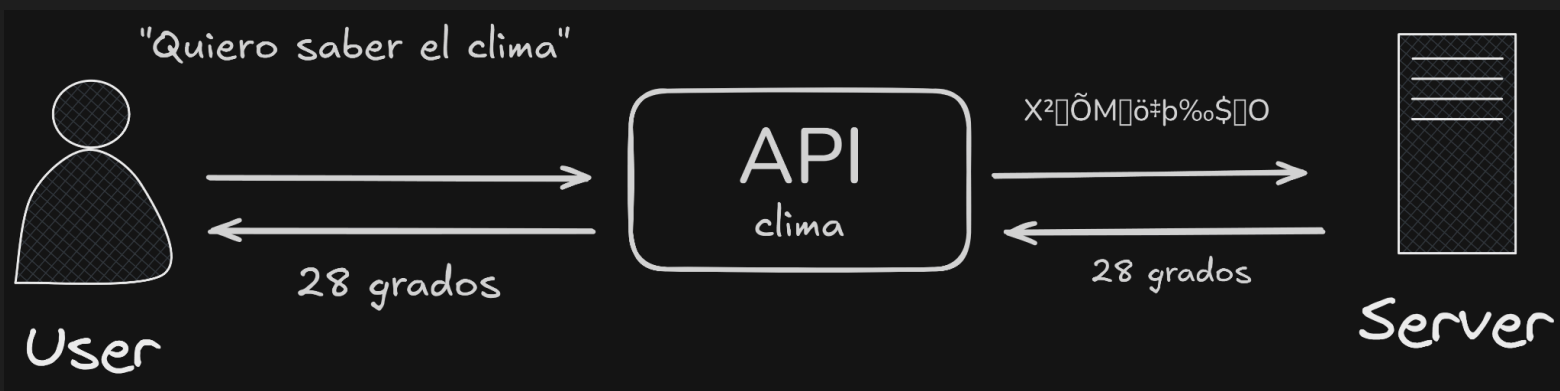
```
// separa un array
```

```
let arr = ["nombre1", "nombre2", "nombre3"]
```

```
console.log(...arr);
```

```
// nombre1, nombre2, nombre3
```

APIs



Un **API** es un conjunto de definiciones y protocolos que permiten que diferentes aplicaciones se comuniquen entre sí.

Su función principal es **exponer funcionalidades** o **datos** de un software para que otros programas puedan interactuar con él, sin necesidad de conocer su implementación interna.

Date

```
const fecha = new Date() -> Devuelve un número en milisegundos  
que quiere decir la fecha actual  
fecha.getHours()  
fecha.getMinutes()  
fecha.getSeconds()
```

Drag and Drop

Eventos del objeto

```
dragstart  
drag  
dragend
```